

英特尔杯嵌入式项目成果论文方向匹配与选题建议深度研究报告

执行摘要

你们的项目来自英特尔杯嵌入式方向竞赛：该赛事采用开放式命题，参赛队需要基于指定的英特尔嵌入式平台，自主提出课题并在限定周期内完成可运行原型与报告。¹ 2026 年竞赛主题为 “AI application and innovation (AI 应用与创新)”，赛程覆盖 2026-03-31 到 2026-06-30，并要求提交项目选题与最终报告/系统。² 同时，竞赛明确要求作品基于指定硬件平台：2026 年将提供基于 Intel® Core Ultra 5 Processor 225U 的 DK-2500 开发套件。³

如果你们的核心成果确实如你此前描述的那样：围绕 **PCM Skills 架构**（可插拔技能/工具调用/上下文编排框架）、配套 **ARG (Agent Runner/Runtime)** 与后端系统工程化，那么最“可论文化”的研究方向通常不是“做了一个能跑的 Demo”，而是把项目升级为**可复现实验的系统研究**：在嵌入式/边缘资源约束（时延、功耗、内存、上下文窗口/Token 预算）下，提出可解释、可治理的 agent 架构或关键算法，并用对照实验验证其在真实平台上的收益。⁴

本报告给出 5 个高匹配且可投稿的论文方向，并对每个方向说明：研究问题、可主张的贡献类型、建议实验与指标、适配投稿渠道。综合发表可行性与你们现有工作线索，优先推荐两条主线选题：

- **首选（系统+方法结合）**：面向嵌入式/边缘 LLM Agent 的“资源感知上下文编译 + 技能路由 (PCM) + 可恢复执行 (Runner/ARG)”体系（偏系统论文/嵌入式软件与边缘计算）。⁵
- **次选（安全与治理）**：工具调用/技能市场（含 MCP）场景下的安全威胁与防护（Prompt injection、越权工具调用、审计与权限证明），尤其适合“嵌入式设备可控外设/执行动作”的风险建模。⁶

假设与未指定项

由于你在本轮问题中未提供项目的“功能场景、系统边界、关键创新点、实验数据”，以下判断基于可验证公开信息与合理工程假设，需你对照实际情况选择最贴近的一条（或两条组合）。

- 假设你们的成果包含：
- PCM Skills：将工具/技能以统一接口注册、描述，并在运行时做技能选择/路由与上下文编排（类似工具调用流程，但强调可插拔与可治理）。⁷
- ARG/Runner：支持多轮模型调用 + 工具执行循环 + 状态管理（可恢复/可取消/可观测）。⁸
- 假设你们部署目标是边缘/嵌入式平台（英特尔杯指定平台），因此论文需要体现“资源约束与真实平台测量”。⁹
- 未指定但会显著影响选题的关键变量：
- 是否“离线/端侧推理为主”还是“端云协同”（会决定论文更像 edge systems 还是 cloud-edge orchestration）。¹⁰
- Skills 是否涉及“可改变外部状态的动作”（如控制硬件、写入数据库、网关调用等），这会决定安全论文的强度与必要性。¹¹
- 你们是否做了模型压缩/加速、功耗优化、或专用硬件利用（这会决定是否可投“边缘高效推理/体系结构”方向）。¹²

竞赛语境与“论文化”要求

英特尔杯嵌入式竞赛强调“开放命题 + 指定平台 + 原型交付”。官方介绍指出：参赛队在指定平台上自主提出题目，完成工作原型并撰写报告，应用可覆盖家庭、汽车、医疗、工业等行业场景。¹³ 2026 年竞赛主题为 AI 应用与创新，赛程为 2026-03-31 至 2026-06-30，并设置项目选题提交与最终报告提交节点。²

从“竞赛成果”到“学术论文”，关键差异通常在于：

- 竞赛更看重可用系统原型；论文更看重**可复现的研究问题、对照方法、与可量化的系统性结论**。这与嵌入式/边缘领域主流投稿口径一致：例如 TECS 的范围明确包含嵌入式软件设计分析、体系结构与加速器、低功耗、可靠性与安全等主题。¹⁴
- 边缘计算/嵌入式软件顶会（如 SEC、EMSOFT、SenSys 等）普遍关注资源管理、可靠性、安全与隐私、面向边缘/IoT 的系统与工具链等主题，这与“嵌入式 Agent 框架/运行时”天然契合。¹⁵

因此，你们项目若要发论文，应尽量把成果表述为下面三类之一（或组合）：

- 1) **系统架构贡献**：提出可落地的 PCM+ARG 架构，解释其模块边界、接口与治理机制，并给出真实平台数据。¹⁶
- 2) **关键方法贡献**：围绕“技能路由/上下文编译/工具选择/预算分配”提出算法或策略改进，并将其嵌入系统验证收益。¹⁷
- 3) **安全与可信贡献**：在工具调用或 MCP 工具生态下做系统威胁建模、攻击复现与防护机制，并量化安全与开销权衡。¹⁸

适合你们项目的论文方向候选

下面给出 5 个候选论文方向。每个方向都按“研究问题 → 可能贡献 → 实验建议 → 适配投稿口径”组织，并在末尾标注（优先级 / 实现难度）。

面向嵌入式/边缘 LLM Agent 的资源感知 PCM：技能路由与上下文编译

研究问题（可写成论文主命题）

在嵌入式平台上，LLM agent 工具调用需要把“工具/技能描述 + 任务上下文 + 检索证据”打包进有限上下文窗口，且必须满足低时延/低功耗/可靠执行。能否设计一种“PCM 控制面”，在给定 token/延迟预算下，动态选择最小技能子集并编译上下文，从而在不牺牲任务成功率的前提下显著降低开销？（你可把这一点视为对工具调用流程的“系统化预算编译扩展”。）¹⁹

你们可能主张的贡献类型

- 提出 PCM 控制面架构：Skill Registry + Router + Context Compiler + Budget/Policy，解释与 agent/workflow 编排框架的边界（例如 LangGraph 提供 stateful workflow 的基础设施，但不替你决定具体 prompts/架构）。²⁰
- 提出“预算约束下的技能选择/上下文编译”策略：可用启发式（如风险/成本/相关性加权）或学习型路由器，并提供可审计 rationale。
- 提供真实平台评测：在 DK-2500（或竞赛指定板卡）上测量端到端延迟、功耗、token 消耗、任务成功率，体现嵌入式约束。²¹

建议实验与指标（论文可复现）

- Baseline：
- “全量工具注入”的朴素工具调用（对照 OpenAI 工具调用 5 步流程中的第 1 步：一次性把工具都给模型）。

- “固定工具集”的规则方案（不随意图变化）。
- Metrics：成功率/完成率、P50/P95 延迟、token 数、能耗（如 RAPL/外接功耗计）、工具调用次数、失败重试次数。²³
- 消融实验：router 关闭/编译器关闭/预算策略不同等。

更容易写出“研究味”的切入点

把“上下文编译”形式化为一个约束优化问题：在 token/延迟预算下最大化“任务价值”，并用可解释日志证明其选择合理。

（优先级：高 | 实现难度：中）

可恢复与可观测的 ARG/Runner：面向长链路工具循环的嵌入式执行语义

研究问题

对工具调用 agent 来说，“一次请求”经常演变成“多轮模型调用 + 多次工具执行 + 异常处理”。如何在嵌入式/边缘环境中把 agent 运行时做成**可中断、可恢复、可回放**的执行语义，并提供可观测事件流（便于调试与评估）？这一思路与当前主流 agent SDK 中“Runner”抽象一致：Runner 提供同步、异步、流式执行，并能导出 tracing 事件。²⁴

可能贡献

- 给出 Runner 的状态机与事件模型（model_call、tool_call、retrieval、router_decision、guardrail 等），并说明与现有框架相比差异点。OpenAI Agents SDK 的 tracing 文档明确把 Runner 与工具调用、guardrails 等包裹在 tracing spans 中，可作为你们论文“对照标准”与术语参考。²⁵
- 在嵌入式场景下给出“轻量持久化事件日志 + 失败恢复策略”，并定量展示提升（例如崩溃注入下的恢复成功率、错误定位速度、现场回放一致性）。

建议实验

- 故障注入：kill 进程/网络抖动/工具超时/模型超时；比较“无事件日志”与“事件溯源+回放”的恢复能力。
- 可观测性：对照 OpenTelemetry/trace span 规范（或追踪平台），展示一次 run 的端到端时间线。²⁶

适配投稿口径

这类工作非常贴合嵌入式软件与边缘系统中的“可靠性、性能分析、QoS 管理、fault tolerance”等主题。²⁷

（优先级：中 | 实现难度：中-高）

端侧 RAG 与混合检索：在资源约束下提升事实性与可验证性

研究问题

RAG 通过把检索到的外部证据注入生成过程来提升知识密集型任务表现，是一种经典路线：原始 RAG 工作将参数化记忆（LLM）与非参数化记忆（向量检索索引）结合，缓解纯生成模型的知识更新与事实性问题。²⁸ 在嵌入式平台上，RAG 面临额外约束：存储受限、索引构建成本、检索时延、以及把证据塞进上下文的 token 成本。能否提出“结构化过滤 + 语义检索”的混合策略，并与 PCM 的上下文编译联动，以最小 token 达到更高可验证性？

可能贡献

- 系统贡献：端侧轻量索引组织、缓存策略、以及“证据选择/压缩”的上下文编译模块。

- 方法贡献：针对嵌入式指标（延迟/功耗/token）设计 retrieval+compression 的联合策略。
- 评估贡献：使用 RAG 评估流程（如检索命中率、引用一致性、事实性）对比不同 pipeline。 29

建议实验

- Baseline: Naive RAG（固定 top-k）、向量-only RAG；可选加入 GraphRAG 对比（若你们有结构化知识图谱/关系抽取）。 30
- Metrics: 答案正确率/事实一致性、引用证据覆盖率、检索 P95 延迟、token 成本；嵌入式侧增加能耗与内存峰值。 31

（优先级：中 | 实现难度：中）

工具调用/技能市场的安全与可信：Prompt Injection、越权工具与审计证明

研究问题

在工具调用系统里，LLM 的输出会驱动下游组件执行动作，因此 OWASP 将 Prompt Injection、Insecure Output Handling、Model DoS、Supply Chain 等列为 LLM 应用关键风险。 11 如果你们 PCM/ARG 支持“技能市场/插件化工具”，安全边界就更复杂：工具描述可能不可信、工具可能是任意代码执行。MCP 规范本身也强调工具代表任意代码执行，必须以谨慎策略处理。 32 近期研究已经开始对 MCP 的协议层安全做系统分析，并指出潜在协议级漏洞与攻击面。 33

你们可能形成的论文贡献

- **威胁模型 + 攻击复现**：在嵌入式工具调用 agent 场景中复现 indirect prompt injection、工具描述投毒、越权工具选择等，参考“prompt injection 影响工具选择”的研究线索与基准。 34
- **防护机制**：
 - 工具权限模型（capability/scopes）、显式人类确认、风险分级（只读 vs 改变状态）、输出净化与参数校验。 35
- **审计链**：每次工具选择为何发生、参数如何生成、是否越权、是否触发策略拒绝。
- **权衡量化**：安全增强带来的成功率下降/延迟开销/交互成本。

适配投稿口径

可投向“嵌入式安全/系统安全/可信 AI 工程”方向：嵌入式软件安全与韧性是 EMSOFT 的明确主题之一；边缘计算 CFP 也把安全与隐私作为重点。 27

（优先级：中-高 | 实现难度：中-高）

面向嵌入式的高效 LLM 推理与端边协同：把“平台优势”写成论文贡献

研究问题

如果你们在英特尔平台上做了显著的推理加速（量化、蒸馏、KV cache 优化、流水线并行、端边协同卸载等），则可以把论文主线放在“边缘高效 LLM 推理与系统优化”。该方向已有系统性综述与边缘 LLM 研究积累，可支撑扎实 related work。 36 NeurIPS 也曾设置 Edge-Device LLM Competition，强调边缘设备资源受限带来的挑战，说明此方向具备学术关注度。 37

可能贡献

- 平台测量：在 DK-2500 上对不同模型大小/量化方案进行时延-能耗-质量三维评估。 9
- 系统优化：把 agent 系统的瓶颈拆解为“检索 + 生成 + 工具调用”的组合开销，并提出端边协同策略（边缘推理+云端重推理 fallback 等），对应 SEC 的 edge-cloud interface、resource management 等话题。 38

(优先级：低-中 | 实现难度：高)

方向对照表与推荐排序

下表把“你们已有工作线索（PCM/ARG/后端）”映射到不同论文方向的可发表性要素。若你们当前缺少某一系列能力，可视为后续补齐的最小差距。

方向	与你们 PCM/ARG 的天然匹配度	需要的关键材料	最像的“论文类型”	典型对标/引用体系	推荐优先级 难度
资源感知 PCM (技能路由+上下文编译)	很高	Router/Compiler 设计、预算策略、真实平台测量、baseline 对比	系统+方法 (edge/embedded agent systems)	工具调用流程、stateful workflow 框架、边缘 LLM 系统综述 ³⁹	高 中
Runner/ARG 可恢复执行语义	高	状态机、事件溯源、故障注入、可观测 tracing	系统软件/运行时	Runner 抽象与 tracing spans ⁸	中 中-高
端侧 RAG + 混合检索 + 证据最小化	中	数据集/知识库、检索 pipeline、证据选择策略、事实性评估	信息检索+系统	RAG 原始论文与综述 ⁴⁰	中 中
安全与治理 (Prompt Injection / 工具越权 / MCP 安全)	中-高 (若有外设控制/可变状态工具)	威胁模型、攻击复现、防护设计、开销权衡	系统安全/可信 AI	OWASP LLM Top 10、MCP 工具安全与相关研究 ⁴¹	中-高 中-高
高效推理/端边协同	取决于你们是否做了推理优化	模型/推理优化细节、功耗/时延测试、质量指标	系统优化/边缘智能	edge LLM survey、edge-device LLM 研究 ⁴²	低-中 高

综合建议：如果你们目前的工程已经包含“技能注册/路由/上下文编排 + agent runner + 后端服务”，但尚未有大幅推理加速工作，那么最稳的发表方向是“**资源感知 PCM（主） + Runner 可恢复执行（辅）**”的系统论文：主线讲“如何在嵌入式资源约束下把工具库做大、把上下文做小、把执行做稳”，副线用事件溯源与可观测性完成工程闭环。 ⁴³

论文叙事结构与实验设计建议

推荐的“主线论文”结构模板

更贴近嵌入式/边缘系统论文的写法可采用以下叙事：

- Problem：嵌入式平台资源约束下，工具库规模与上下文预算冲突，导致延迟、能耗、失败率上升；且工具调用是多步流程，需要系统侧执行与回注。⁴⁴
- Design：提出 PCM (Router+Compiler+Policy+Budget) 与 Runner (state+events+recovery)；明确接口与扩展点。⁴⁵
- Implementation：实现于 DK-2500（或竞赛指定板），说明软硬件环境与约束。²¹
- Evaluation：对照 baseline + 消融；同时评估性能（延迟/能耗）、质量（成功率/正确性）、稳定性（故障注入恢复），必要时加入安全评估（prompt injection/越权工具）。⁴⁶

实验设计的“最低可发表门槛”

为了把竞赛成果变成论文，建议至少补齐以下三类实验（你可以把它们视为验收清单）：

- **规模实验**：工具/技能数量从小到大增长（例如 10/50/200/1000），比较“全量注入”与“动态路由+编译”的 token 与延迟增长曲线。工具调用基准与工具学习领域已有 ToolBench/StableToolBench 等体系，可以作为选取任务与构造工具库的参考。⁴⁷
- **资源实验**：在嵌入式平台上测量 P50/P95 延迟与能耗，体现 edge LLM 系统关注的核心约束。⁴⁸
- **鲁棒性实验**：工具超时、执行失败、模型输出异常等；Runner 的事件溯源与恢复策略是否显著降低失败成本。²⁵

如果你们想走“安全方向”，则增加：

- **攻击与防护实验**：基于 OWASP LLM Top 10 选取 prompt injection、insecure output handling 等攻击面，验证是否导致工具越权或副作用工具被触发，并量化防护开销。⁴⁹

选题决策流程图（Mermaid）

flowchart TD

```
A[竞赛成果清点] --> B{核心亮点是什么?}
B -->|技能路由/上下文预算| C[方向：资源感知 PCM]
B -->|可恢复执行/可观测| D[方向：Runner/ARG 运行时]
B -->|离线知识/检索效果| E[方向：端侧 RAG]
B -->|外设控制/插件生态| F[方向：工具调用安全与治理]
B -->|推理加速/功耗优化| G[方向：高效推理与端边协同]

C --> H[需要：工具规模实验+token/延迟曲线]
D --> I[需要：故障注入+回放一致性]
E --> J[需要：事实性评估+检索时延]
F --> K[需要：威胁模型+攻击复现+防护权衡]
G --> L[需要：质量-时延-能耗三维评估]
```

潜在投稿渠道与策略建议

这里给出“与上述方向最匹配”的投稿渠道类型（不是穷举），并解释为什么合适。考虑你们项目来自嵌入式竞赛，建议采用“两步走”：先投短文/Workshop/Demo 快速形成可公开成果，再打磨成更完整的期刊/顶会长文。

国际嵌入式/边缘系统方向

- **ACM TECS（期刊）**：范围覆盖嵌入式软件设计分析、加速器、低功耗、安全可靠、embedded ML 等，非常适合“嵌入式 agent 系统/运行时/资源预算”类论文。⁵⁰（推荐：中 | 难度：高）
- **EMSOFT（ESWEEK 体系）**：主题覆盖嵌入式分布式系统、资源分配与性能分析、网络安全/安全韧性、测试验证等，适合“Runner 可恢复执行 + 工具调用可靠性/安全”路线。⁵¹（推荐：中 | 难度：高）
- **ACM/IEEE SEC（边缘计算）**：CFP 强调 edge-cloud interface、resource management、reliability、security/privacy 等，非常适配“PCM 在边缘的资源预算管理”与“端边云协同”。³⁸（推荐：中 | 难度：高）
- **SenSys（合并了 SenSys/IPSAN/IoTDL 社区）**：topics 明确包含 Embedded AI、以及“Large language models and multi-modal large models for embedded systems”“distillation to edge”等，若你们能体现嵌入式平台测量与真实部署，会非常贴合。⁵²（推荐：中 | 难度：很高）

国内期刊/会议方向（更贴近“工程化智能体框架”）

- **《软件学报》相关专刊**：例如“大模型与软件工程”专刊的征文范围明确包含“GenAI 赋能智能体或大模型支撑框架与系统的可靠性研究、可解释性与可信性研究”等主题，这与 PCM/ARG 工程化非常对口。⁵³（推荐：高 | 难度：中）
- **CCF 嵌入式系统相关会议（ESTC 等）**：征文范围覆盖嵌入式体系结构、实时系统、嵌入式操作系统与中间件、边缘计算与物联网等方向，适合作为首篇中文论文或阶段成果。⁵⁴（推荐：中 | 难度：中）

策略建议（与你们竞赛时间线对齐）

竞赛本身在 2026-03-31 至 2026-06-30 期间完成系统，并要求随后提交报告与参评材料。² 论文准备可以按以下节奏压缩：

- 在竞赛交付完成后 4-6 周内补齐“对照实验 + 消融 + 平台测量”，形成可投稿的第一版；
- 若选择安全方向，优先补齐 OWASP 风险清单对照与攻击复现（至少 2-3 类攻击面），这样论文更像“系统安全研究”而非经验总结。⁴⁹

（优先级：高 | 实现难度：中）

结论性推荐

在你未提供更具体项目细节的前提下，最稳、最贴合你们已有工作线索的论文方向是：

- **方向 A：资源感知 PCM（技能路由 + 上下文编译）+ 嵌入式真实平台评测**（主方向，目标系统/嵌入式/边缘计算投稿口径）。⁵⁵
- **方向 B：Runner/ARG 的可恢复执行语义与可观测性（事件溯源）**（作为论文第二贡献点，强化“工程可落地与可复现”）。²⁵

如果你们系统确实涉及“插件生态/技能市场/MCP 工具接入”或者“可改变状态的工具（控制硬件/调用外部系统）”，则强烈建议把**安全与治理**作为论文关键章节甚至主线之一，因为该方向已有明确行业风险框架与新近研究支撑，容易形成可发表的系统性结论。⁶

-
- 1 13 Intel Cup Undergraduate Electronic Design Contest – Embedded System Design Contest
https://nuedc.sjtu.edu.cn/en/content.aspx?flag=21&info_lb=40
- 2 Intel Cup Undergraduate Electronic Design Contest – Embedded System Design Contest
https://nuedc.sjtu.edu.cn/en/show.aspx?flag=20&info_id=1309&info_lb=20
- 3 9 21 英特尔杯大学生电子设计竞赛嵌入式AI专题赛
https://nuedc.sjtu.edu.cn/cn/show.aspx?flag=1&info_id=1311&info_lb=1
- 4 10 15 23 38 ACM/IEEE Symposium on Edge Computing
https://acm-ieee-sec.org/2025/call_for_papers.php?utm_source=chatgpt.com
- 5 14 43 50 55 ACM Transactions on Embedded Computing Systems
https://dl.acm.org/journal/tecs/about?utm_source=chatgpt.com
- 6 11 18 41 46 49 OWASP Top 10 for Large Language Model Applications
https://owasp.org/www-project-top-10-for-large-language-model-applications/?utm_source=chatgpt.com
- 7 17 19 22 39 44 Function calling | OpenAI API
https://developers.openai.com/api/docs/guides/function-calling/?utm_source=chatgpt.com
- 8 24 Running agents - OpenAI Agents SDK
https://openai.github.io/openai-agents-python/running_agents/?utm_source=chatgpt.com
- 12 42 A survey of edge efficient LLMs and techniques
https://www.sciencedirect.com/science/article/abs/pii/S1574013725000310?utm_source=chatgpt.com
- 16 27 51 EMSOFT: CFP
https://esweek.org/emsoft_cfp/?utm_source=chatgpt.com
- 20 45 LangGraph overview - Docs by LangChain
https://docs.langchain.com/oss/python/langgraph/overview?utm_source=chatgpt.com
- 25 26 openai-agents-python/docs/tracing.md at main
https://github.com/openai/openai-agents-python/blob/main/docs/tracing.md?utm_source=chatgpt.com
- 28 40 Retrieval-Augmented Generation for Knowledge-Intensive ...
https://arxiv.org/abs/2005.11401?utm_source=chatgpt.com
- 29 Retrieval-Augmented Generation for Large Language ...
https://arxiv.org/abs/2312.10997?utm_source=chatgpt.com
- 30 A Graph RAG Approach to Query-Focused Summarization
https://arxiv.org/abs/2404.16130?utm_source=chatgpt.com
- 31 36 48 A Review on Edge Large Language Models: Design ...
https://dl.acm.org/doi/full/10.1145/3719664?utm_source=chatgpt.com
- 32 Specification
https://modelcontextprotocol.io/specification/2025-11-25?utm_source=chatgpt.com
- 33 Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents
https://arxiv.org/abs/2601.17549?utm_source=chatgpt.com
- 34 Prompt Injection Attack to Tool Selection in LLM Agents
https://arxiv.org/html/2504.19793v2?utm_source=chatgpt.com

35 OWASP LLM02: Insecure Output Handling

https://genai.owasp.org/llmrisk2023-24/llm02-insecure-output-handling/?utm_source=chatgpt.com

37 Edge-Device Large Language Model Competition

https://neurips.cc/virtual/2024/competition/84790?utm_source=chatgpt.com

47 ToolLLM: Facilitating Large Language Models to Master ...

https://arxiv.org/abs/2307.16789?utm_source=chatgpt.com

52 2025 ACM SenSys Call for Papers

https://sensys.acm.org/2025/cfp.html?utm_source=chatgpt.com

53 大模型与软件工程（截稿时间：2025年9月8日）

https://www.jos.org.cn/jos/news/view/20250317095413001?utm_source=chatgpt.com

54 投稿指南

https://easychair.org/cfp/CCFESTC2021?utm_source=chatgpt.com